

High-Performance Matrix Multiplication on Graphics Processing Units (GPUs)

PhD Course Project Report: GPU-Accelerated Computing for AI and Computer Vision

Fabio A. Ciraci

GitHub Repository: GPU-Matrix-Multiplication-Project

July 13, 2026

Abstract

Matrix multiplication ($C = A \times B$) serves as a fundamental building block in modern scientific computing, deep learning, and computer vision. Achieving peak theoretical performance on modern Graphics Processing Units (GPUs) requires overcoming severe memory bandwidth and instruction-level bottlenecks. This report details the implementation, optimization, and evaluation of various parallel matrix multiplication. The firsts test runned are naive CPU implementations of Matrix Multiplication algorithms, while the last is the implementation of a CUDA specific multiplication via the CuPy lib. All the code is in python. LLMs were used to support code writing and the report.

1 Introduction

Matrix Multiplication is a core operation in numerous computational tasks, including training Large Language Models (LLMs), climate modeling, and scientific simulations. The General Matrix Multiply (GEMM) operation, which computes the product of two dense matrices, is particularly critical due to its high computational intensity and its role in linear algebra routines that underpin many machine learning algorithms.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x & y \\ z & w \end{bmatrix} = \begin{bmatrix} ax + bz & ay + bw \\ cx + dz & cy + dw \end{bmatrix}$$

The Matrix Multiplication algorithm has a time complexity of $O(N^3)$ for naive implementations, where N is the dimension of the square matrices. This makes optimization crucial for large-scale applications, especially when leveraging the parallel processing capabilities of GPUs. In this report, we explore various implementation of matrix multiplication, from simple naive algorithms to more advanced techniques such as tiling and trasposed.

2 Implementation & Optimization Strategies

This section forms the core technical contribution of your report. Describe each implementation variant step-by-step.

Given the interpreted nature of Python, we leverage the Numba library to JIT-compile our matrix multiplication functions for CPU execution.

To JIT-compile a function, we use the `@jit` decorator from Numba. This allows us to write Python code that is compiled to machine code at runtime, significantly improving performance.

2.1 Baseline CPU Implementation

The naive implementation leverages three nested loops to compute the product of two matrices.

```
1 import numpy as np
2
3 @jit(nopython=True)
4 def naive_approach(A, B, C):
5     for i in range(N):
6         for j in range(N):
7             for k in range(N):
8                 C[i][j] += A[i][k] * B[k][j]
```

Listing 1: Baseline Naive CPU Matrix Multiplication in Python

2.2 Exploiting Locality principle

Using the locality principle, we can improve cache performance by accessing data in a more cache-friendly manner. CPUs pre-load the data into cache line by line, in the naive implementation the A matrix is accessed row-wise, while the B matrix is accessed column-wise, which can lead to cache misses. It is then possible to invert the order of the loops to access the B matrix row-wise, which is more cache-friendly and can lead to better performance.

```
1 @jit(nopython=True)
2 def exploiting_locality(A, B, C):
3     for i in range(N):
4         for k in range(N):
5             for j in range(N):
6                 C[i][j] += A[i][k] * B[k][j]
```

Listing 2: Baseline Naive CPU Matrix Multiplication with Locality Principle in Python

2.3 Transposed Approach

In the transposed approach, we first transpose the second matrix B before performing the multiplication. This allows us to access both matrices in a row-wise manner, which is more cache-friendly and can lead to better performance.

```
1 @jit(nopython=True)
2 def transposed_approach(A, B, C):
3     for i in range(N):
4         for j in range(N):
5             for k in range(N):
6                 C[i][j] += A[i][k] * B[j][k]
7
8 ## In the main code
9 B_t = np.transpose(B).copy()
10 transposed_approach(A, B_t, C)
```

Listing 3: Baseline Naive CPU Matrix Multiplication with Locality Principle in Python

2.4 Tiled Matrix Multiplication (Shared Memory)

In Tiled Matrix Multiplication, we divide the matrices into smaller sub-matrices (tiles) that fit into the shared memory of the GPU. This approach reduces the number of global memory accesses, which are slower than shared memory accesses, and allows for better utilization of the GPU's parallel processing capabilities.

```
1 @jit(nopython=True)
2 def tiling_approach(A, B, C, block_size):
3     for ih in range(0, N, block_size):
```

```

4         for kh in range(0, N, block_size):
5             for jh in range(0, N, block_size):
6                 # min in necessary in case the matrix is smaller than the sum of the
7                 # the block and the already reached position
8                 for i in range(ih, min(ih + block_size, N)):
9                     for k in range(kh, min(kh + block_size, N)):
10                         for j in range(jh, min(jh + block_size, N)):
11                             C[i][j] += A[i][k] * B[k][j]

```

Listing 4: Tiled Matrix Multiplication

2.5 GPU Optimization with CuPy

Specialized GPU libraries like CuPy can be used to perform matrix multiplication on the GPU with minimal code changes.

```

1 import cupy as cp
2
3 A = np.random.rand(N, N).astype(np.float32)
4 B = np.random.rand(N, N).astype(np.float32)
5 C = np.zeros((N, N), dtype=np.float32)
6
7 C = cp.dot(A, B)
8 cp.cuda.Stream.null.synchronize()

```

Listing 5: CuPy Implementation

3 Hardware Architecture

The hardware architecture used for testing and evaluation is both a modern CPU and NVIDIA GPU. The local CPU is an Intel Core i7-12700H with 14 cores (6 performance cores and 8 efficiency cores) and a base clock of 2.3 GHz. Local GPU for testing is an NVIDIA RTX 4050 Laptop GPU with 6GB of GDDR6 memory, 2560 CUDA cores, and a base clock of 1.5 GHz.

4 Experimental Setup

Experimental tests in *Table 1* shows interesting results, the JIT compiler enhanced the results of the naive approaches, in particular the Naive, Locality Exploitation and Transpose Naive approaches. In case of not using the JIT compiler the tiling approach is the best, but when using the JIT compiler the Locality performs better. This may be due to the fact that the JIT compiler is not capable of translating into low-level code the tiling approach, while it is capable of optimizing the locality approach.

The results obtained by the **CuPy** library are not reported in the table, but they outperform all the other approaches.

In particular the Execution time is **69.95 ms**, with a caveat, the CuPy library has a transfer time of **111.86 ms**, which is the time it takes to transfer the data from the CPU to the GPU and back. When transferring very huge matrix this cost is negligible, but when transferring small matrix this cost is very high and can be a bottleneck.

Table 1: Results of naive approaches with and without JIT-compiler ($N = 512$)

Configuration/Approach	Without JIT (ms)	With JIT (ms)
Naive	45014.81	952.77
Locality Exploitation	40713.50	97.33
Transpose Naive	40872.50	116.37
Tiling (Block Dimension)		
Tile 8	42968.00	41921.99
Tile 16	42771.56	40756.99
Tile 32	42471.79	40455.29
Tile 64	39289.19	39580.47
Tile 128	39916.33	40033.89
Tile 256	39080.68	40148.99
Tile 512	38009.71	40938.10
Tile 1024	39274.97	40152.30
Tile 2048	38422.95	40186.30
Tile 4096	39302.94	40590.50

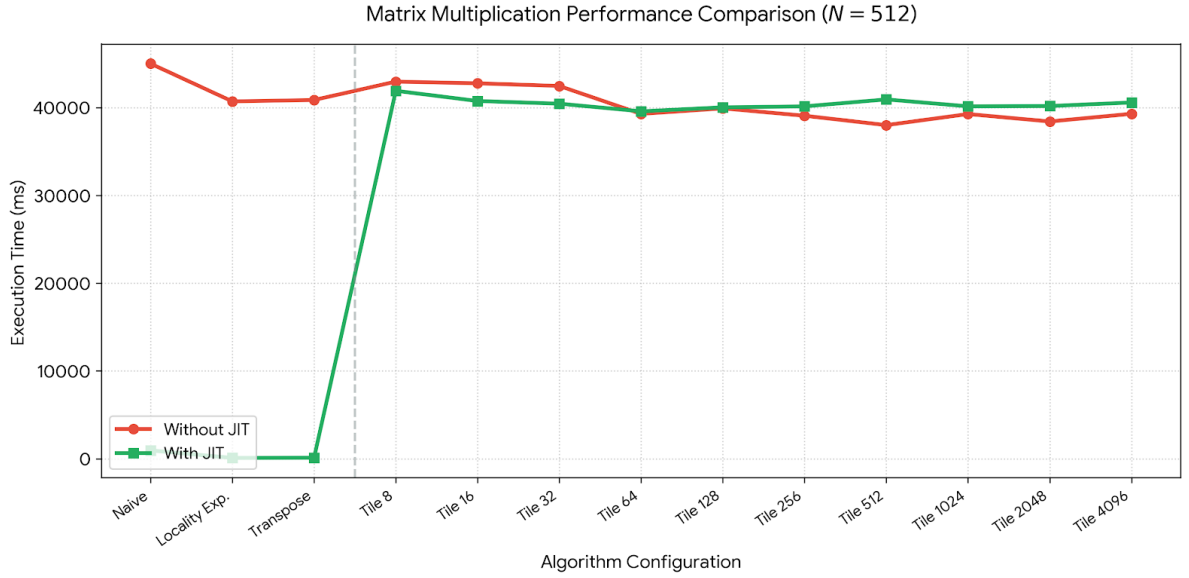


Figure 1: Comparison of temporal performance (ms) with and without JIT compiler. On the left are the classic approaches, on the right is the behavior of Tiling as the block size varies.

5 Discussion

The results shows that on local systems the matrix multiplication is a non-trivial problem, especially when using high-level languages such as Python. Even though the transposed approach is more cache-friendly the `np.transpose().copy()` function creates a copy of the matrix, thus not really creating a new transposed version of the matrix, eliminating the benefits of the transposed approach.

The JIT compiler is able to optimize the code and make it run faster, but it is not able to optimize the tiling approach, which is the best approach when not using the JIT compiler. The CuPy library is able to optimize the matrix multiplication on the GPU, but it has a transfer time that can be a bottleneck when transferring small matrix.

6 Conclusion and Future Work

This report has explored various implementations of matrix multiplication, from naive CPU approaches to optimized GPU techniques using CuPy. The findings highlight the significant performance gains achievable through JIT compilation and GPU acceleration, while also emphasizing the importance of considering data transfer overheads in GPU computations. Future work will could be interesting to explore more advanced GPU optimization techniques in particular on multi-GPU systems, and to investigate the impact of different matrix sizes and shapes on performance.